

LOGPRESS · 학습 진행 발표

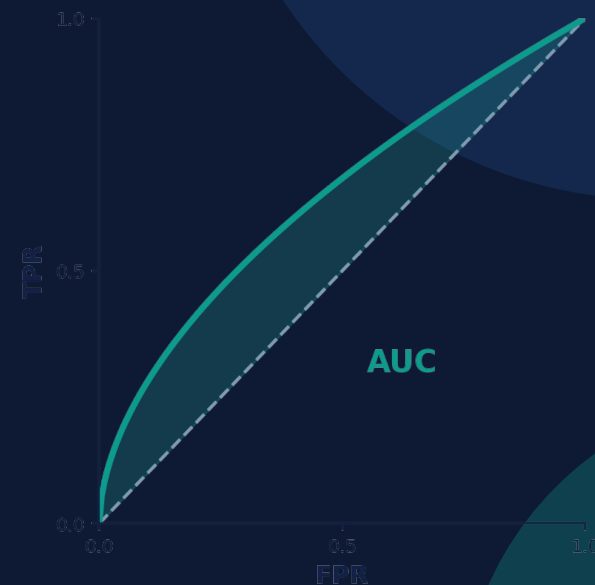
LogPress 프로젝트 학습 진행 발표

기초 역량 학습 + AUC 신호 판정 방법론

PART A LogPress 기초 학습

PART D AUC 신호 판정

공동 발표 · 2026. 08. 02



AGENDA

발표 구성



PART A

LogPress 기초 학습

- 1 단계 · Python 문법 — 학습 완료
- 2 단계 · PyTorch 기초 — 학습 진행 중
- 프로젝트 코드로 직접 연결되는 문법 중심

발표 · A



PART D

AUC 신호 판정

- 개념 · 수식 이해 (ROC / AUC / 순위 공식)
- 코드 구현 — 가설 1·2·3
- 순열검정 · p 값 · GO/NO-GO 판정

발표 · D

PART A

PART A

LogPress 기초 학습 진행

담당 A · Python 문법 · PyTorch 기초

학습 로드맵 — 현재 위치

1

Python 문법

학습 완료

100%

공식 한국어 자습서의 필요한 단원을 순서대로 학습
로그 처리 · AI 모델 코드에 직접 연결되는 문법 중심

2

PyTorch 기초

진행 중

약 50%

Tensors · Quickstart 까지 학습 완료
다음 — PyTorch 기초를 계속 익히며 프로젝트 코드 이해 확장

Python 문법 — 학습 완료

단원	학습 내용	프로젝트에서 활용되는 곳
3 장	숫자 · 문자열 · 리스트	로그 문자열과 토큰 목록을 저장 · 처리
4 장	if · for · 함수	로그 판별 · 반복 처리 · 기능을 함수로 분리
5 장	리스트 · 튜플 · 딕셔너리	모델 입력값 · 캐시처럼 여러 값을 묶은 구조 처리
6 장	모듈과 import	torch · transformers 등 외부 도구를 불러와 사용
7 장	출력과 파일 읽기	로그 파일을 읽고 처리 결과를 확인
8 장	예외 처리	파일 · 모델 실행 중 오류를 안전하게 확인
9 장	클래스	DynamicCache · LogPress 구조 이해의 기초
12 장	가상환경과 pip	프로젝트별 패키지 · 버전을 독립적으로 관리

PyTorch 기초 — 학습 진행 중

Tensors

AI 가 숫자 데이터를 저장 · 계산할 때 쓰는 다차원 데이터 구조를 학습

연결 KV Cache 의 크기 · shape · 자료형 · 연산을 이해하는 기초

Quickstart

데이터를 준비해 모델에 넣고 결과를 얻는 PyTorch 전체 사용 흐름을 학습

연결 Qwen 모델에 로그를 입력하고 출력 · 캐시를 확인하는 코드로 이어짐

다음 학습 PyTorch 기초 내용을 계속 익히며 프로젝트 코드에 필요한 이해를 넓혀갈 예정

PART D

PART D

AUC 신호 판정

담당 D · 개념 → 수식 → 코드 → 검정

무엇을 판정하나 — 원인 줄에 신호가 있는가

표준 방식

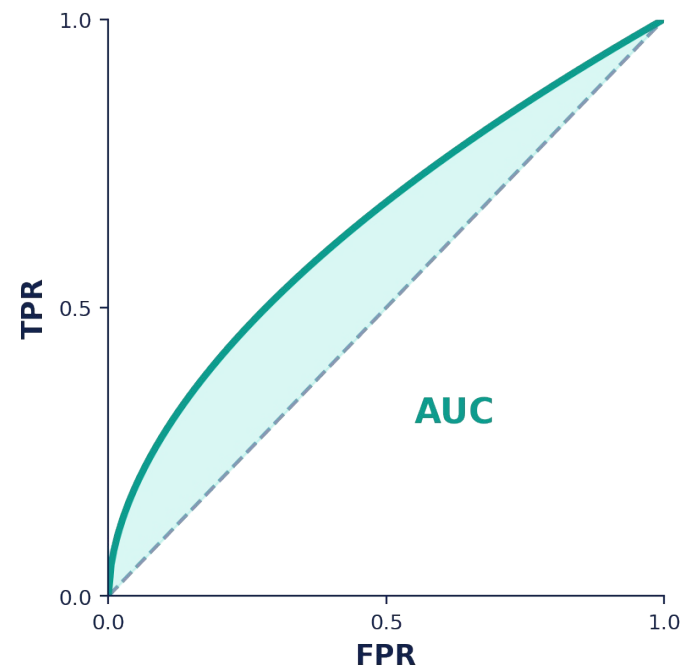
혼동행렬 TP·FP → TPR·FPR → ROC 곡선 → AUC (0~1, 구별력)

막힘

킬스위치엔 모델 (예측 점수) 이 없어 TP·FP 를 셀 수 없다

재정의

“ 위치라는 score 로 양성 / 음성이 갈리는가 ” 를 보는 문제로 전환



TP 정답 (양성) FP 오답 (양성 예측) $TPR = TP / (TP + FN)$ $FPR = FP / (FP + TN)$

TP/FP 없이 AUC 구하기 — 순위 (rank) 공식

$$\text{AUC} = \left(R_{\text{pos}} - n_{\text{pos}}(n_{\text{pos}}+1) / 2 \right) / \left(n_{\text{pos}} \times n_{\text{neg}} \right)$$

Mann-Whitney U · 전체를 정렬해 순위를 매긴 뒤 양성의 순위 합 R_{pos} 만 사용

1 순서만 본다

roc_auc_score 는 값의 크기가 아니라 순위 (순서) 만 반영한다

2 위치 = score

y_score 에 위치를 넣으면 “위치가 양성 · 음성을 잘 나누나” 를 측정

3 내부 = U

roc_auc_score 가 내부에서 하는 계산이 곧 Mann-Whitney U

코드 구현 — 세 가지 가설

가설 1

전역 위치

position_ratio 를 그대로 score 로 사용 →
roc_auc_score(is_cause, position_ratio)

가설 2

창 (5 분 · template)

node · 5 분격자 · template_id 로 창 구성
→ 창 내 rank/(n-1)

가설 3

창 (1 시간)

node · grid_id_1h 로 창 구성 →
rank/(n-1), is_hardcase 판정

```
df['rank'] = df.groupby(...).cumcount()  
df['n']     = df.groupby(...)['line_id'].transform('size')  
df = df[df['n'] >= 2]  
y_score = df['rank'] / (df['n'] - 1)  
roc_auc_score(y_test, y_score)
```

예시 결과

가설 1

AUC 0.554 · p 0.0005

유의하지만 크기는 작음

우연이 아님을 증명 — 순열검정

1

라벨 (y_{test}) 을 무작위로 섞어 위치-정답 관계를 끊는다 = 신호 없는 가짜 상황 (H_0)

2

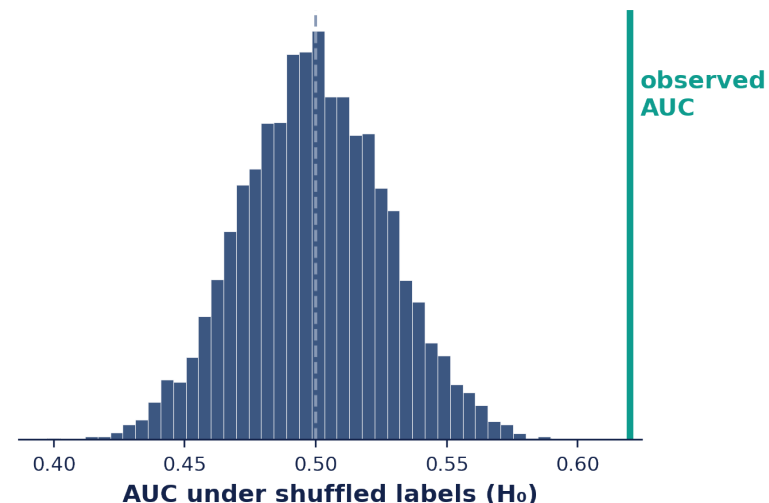
섞은 라벨로 AUC 를 쟀다 — 방금 쟀 `roc_auc_score` 재사용

3

5,000 번 반복 → 우연일 때의 AUC 분포를 만든다

4

실제 AUC 가 그 분포보다 극단적인 비율 = p 값



$$p = (\text{count} + 1) / (n_{\text{perm}} + 1)$$

양측 — $|AUC - 0.5|$ 기준으로 비교

핵심 “5,000 번 반복” 은 진짜 반복 작업 → 여기서는 for 문이 맞다

최종 판정 기준 — GO / NO-GO

GO = $AUC \geq 0.65$ 또는 ≤ 0.35 (크기) **AND** $p < 0.05$ (우연 아님)

크기 조건 — AUC

0.5 에서 얼마나 · 어느 방향으로 벗어났나 · 쓸모 있을 만큼 커야 실제로 신호로 쓸 수 있다.

우연 조건 — p 값

표본이 크면 아주 작은 차이도 유의해진다 (예 : AUC 0.554, p 0.0005). p 값만으론 반쪽 — 크기가 함께 필요.

정리 · 배운 점

핵심 정리

- 1 **pandas 사고** 하나씩 (for) 돌지 않고 전체를 한 번에 처리 — 계산 결과는 `df['이름']` 으로 저장해야 걸러도 짝이 안 깨진다
- 2 **순위 = 신호** `roc_auc_score` 는 순서만 보는 함수 → 위치를 `score` 로 넣어 신호를 판정
- 3 **두 조건** AUC(크기) + p 값 (유의성) 둘 다 있어야 GO
- 4 **반복 학습** 문서를 한 번 썼다고 이해가 끝난 건 아니다 — 사고 흐름을 반복해 말하며 익힌다